

2026학년도 2학기 언어데이터과학

제6강 Python 활용 (1) 수치 연산과 배열

박수민

서울대학교 인문대학 언어학과

2026년 9월 21일 월요일

지난 시간에 배운 것

- 1 데이터는 사물의 종류가 아니라 **증거로 쓰이는 자리**다.
- 2 기술통계량: 중심(45.12 / 33 / 17), 산포(0 / 20 / 33 / 61 / 273), 관계(r)
- 3 요약값 하나는 반드시 무엇인가를 감춘다. 그려 보고, 나누어 보아야 한다.

오늘의 목표

- 1 Jupyter Notebook에서 Python 코드를 실행하고 저장소에 남길 수 있다.
- 2 배열(ndarray)이 리스트와 무엇이 다른지 설명할 수 있다.
- 3 벡터화 연산과 불리언 마스크로 for문 없이 계산할 수 있다.
- 4 5강에서 AWK로 구한 값을 numpy로 다시 구할 수 있다.

오늘의 출발점

5강 마지막 슬라이드에 적어 둔 코드

```
import numpy as np

words = np.loadtxt('data/apicius/index.tsv', delimiter='\t',
                  skiprows=1, usecols=2)
print(words.mean(), np.median(words), words.std(ddof=1))
```

오늘 할 일

이 다섯 줄을 **한 글자씩** 이해한다. 그러면 5강에서 AWK로 며칠에 걸쳐 구한 값이 어떻게 한 줄로 나오는지 알게 된다.

새 데이터를 가져오지 않는다. 4강에서 만든 표와 3강에서 만든 빈도 목록, 이미 가진 파일 두 개로만 진행한다.

사전 준비

터미널 입력 (Codespaces)

```
cd /workspaces/LDS2026
wc -l data/apicius/index.tsv data/apicius/word-freq.txt
pip install numpy
```

실행 결과

```
500 data/apicius/index.tsv
2434 data/apicius/word-freq.txt
2934 total
```

index.tsv가 500행인 것은 머리글 행이 하나 있기 때문이다. 레시피는 499개다. numpy가 이미 깔려 있으면 pip이 “Requirement already satisfied”라고 알려 준다.

1 Python으로 옮겨오기

Jupyter Notebook
왜 numpy인가

2 배열

3 벡터화 연산

4 언어데이터에 적용하기

5 마무리

터미널만으로는 부족해지는 순간

AWK가 잘한 일

- 파일을 한 줄씩 훑으며 걸러내기
- 한 줄짜리 명령으로 즉시 답 얻기
- 큰 파일도 메모리 걱정 없이

AWK가 힘들어한 일

- 값을 모아 두고 두 번 훑기
- 중간 결과를 눈으로 확인하기
- 계산 과정을 남에게 보여 주기

5강의 awk/stats.awk를 떠올려 보자

표준편차 하나를 구하려고 값 499개를 배열에 담고, 평균을 구하고, 다시 처음부터 훑었다.
16줄이 필요했고, 중간에 무엇이 들어 있는지는 보이지 않았다.

오늘부터는 계산을 **한 덩어리씩** 실행하고, 그때마다 결과를 눈으로 본다.

Jupyter Notebook

노트북이란

코드를 **셀(cell)** 단위로 나누어 실행하고, 결과를 코드 바로 아래에 붙여 두는 문서(.ipynb).

좋은 점

- 한 셀씩 고쳐 가며 실험할 수 있다
- 코드와 결과와 설명이 한 파일에 남는다

조심할 점

- 실행 **순서**가 뒤엉킬 수 있다
- 지운 셀의 결과가 메모리에 남는다

커널 (kernel)

셀을 실제로 실행하는 Python 프로세스. 화면 뒤에서 돌아가며 변수를 기억한다.

노트북 만들기

Codespaces에서 할 일

- 1 2강에서 만든 Codespace를 연다. (새로 만들지 않는다)
- 2 Explorer의 [New File...]를 눌러 06-numpy.ipynb를 만든다.
- 3 오른쪽 위 [Select Kernel] → [Python Environments] → Python 3.12.x
- 4 확장 설치를 물으면 [Install]을 누른다. (Python, Jupyter)

저장소 맨 위에 만든다

노트북은 자기가 놓인 폴더를 기준으로 파일을 찾는다. notebooks/ 같은 하위 폴더에 두면 data/apicius/index.tsv가 더 이상 그 경로에 없다.

4·5강의 터미널과 **같은 자리**에서 시작해야 같은 경로를 쓸 수 있다.

첫 셀

노트북 입력 (Shift+Enter로 실행)

```
import os  
os.getcwd()
```

실행 결과

```
'/workspaces/LDS2026'
```

셀의 **마지막 식**은 저절로 표시된다 (여러 값을 함께 보려면 print)

이 결과가 /workspaces/LDS2026이 아니면 노트북의 위치가 잘못된 것이다.

셀은 두 가지다

코드 셀 (Code)

Python 코드를 담고 실행한다. 결과는 셀 바로 아래에 붙는다.

[] 아직 실행하지 않았다

[*] 지금 실행 중이다

[3] 세 번째로 실행되었다

마크다운 셀 (Markdown)

설명을 담는다. 실행하면 서식이 입혀진 글이 된다.

```
## 1. 데이터 읽기  
`index.tsv`의 words 열
```

다시 고치려면 그 글을 두 번 클릭한다.

코드만 있는 노트북은 석 달 뒤의 내가 읽지 못한다

무엇을 왜 하는지를 코드 바로 위에 마크다운 셀로 적어 둔다. 기말 프로젝트 보고서도 이렇게 코드와 설명을 섞어 쓰게 된다.

셀을 만들고, 바꾸고, 지우기

화면에서

노트북 맨 위의 [+ Code]와 [+ Markdown] 단추를 누르거나, 셀과 셀 사이에 마우스를 올렸을 때 나타나는 단추를 누른다. 옮기기·지우기는 셀 오른쪽 위에 뜨는 단추들에 있다.

셀 다루기

- A / B — 위/아래에 새 셀
- M / Y — 마크다운/코드로
- DD — 지우기
- Z — 되돌리기

실행하기

- Shift+Enter — 실행 후 아래로
- Ctrl+Enter — 실행 후 제자리
- Alt+Enter — 실행 후 새 셀

왼쪽 단축키는 Esc를 눌러 **명령 모드**로 나온 뒤에만 듣는다. Enter를 누르면 다시 **편집 모드**다. (macOS의 Alt는 Option)

노트북의 가장 흔한 함정

화면의 순서 $\neq$ 실행의 순서

셀 왼쪽의 [1], [2]는 **실행한 차례**다. 위아래로 오가며 고치다 보면 위쪽 셀이 아래쪽 셀보다 나중에 실행되어 있을 수 있다.

내 노트북이 재현 가능한지 확인하는 법

제출하기 전에 반드시 [Restart] 후 [Run All]을 눌러 본다.

- 커널을 꺾다 켜므로 기억하던 변수가 모두 사라진다.
- 위에서 아래로 한 번에 실행해서 끝까지 가면 재현 가능한 노트북이다.
- 중간에서 멈추면, 남이 실행해도 똑같이 멈춘다.

5강에서 말한 “데이터가 아니라 데이터를 만든 명령을 남긴다”가 노트북에서는 이렇게 실천된다.

순수 Python으로 평균 구하기

노트북 입력

```
1 nums = []
2 with open('data/apicius/index.tsv') as f:
3     next(f) # 머리글 행 건너뛰기 (AWK의 NR > 1)
4     for line in f:
5         nums.append(int(line.split('\t')[2]))
6
7 print(len(nums), sum(nums) / len(nums))
```

실행 결과

```
499 45.124248496993985
```

답은 맞다. 그런데 다섯 줄이 필요했고, 표준편차를 구하려면 여기서 또 늘어난다.

리스트로는 계산이 안 된다

기대

```
[1, 2, 3] + [10, 20, 30]
```

원소끼리 더해 [11, 22, 33]?

실제

```
[1, 2, 3, 10, 20, 30]
```

뒤에 **이어 붙는다.**

다른 연산도 마찬가지다

```
[1, 2, 3] * 2 [1, 2, 3, 1, 2, 3] — 두 배가 아니라 두 번 반복
```

```
[1, 2, 3] + 1 TypeError: can only concatenate list (not  
"int") to list
```

```
[1, 2, 3] * 1.5 TypeError: can't multiply sequence by  
non-int of type 'float'
```

리스트의 +와 *는 **수의 연산**이 아니라 **줄 세우기**다.

그래서 numpy

배열로 바꾸면

```
import numpy as np
```

```
a = np.array([1, 2, 3])
```

```
b = np.array([10, 20, 30])
```

```
a + b
```

```
array([11, 22, 33])
```

```
array([2, 4, 6])
```

a * 2의 결과

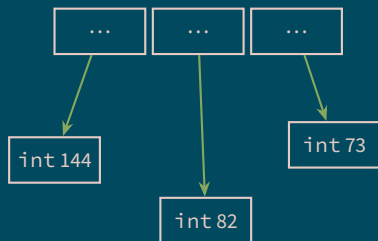
```
import numpy as np
```

np라는 짧은 이름을 붙이는 것은 전 세계가 지키는 관례다. 남의 코드를 읽을 때 np.로 시작하면 numpy라고 바로 알아본다.

벡터화 연산(vectorized operation): 배열 전체에 한 번에 적용되는 원소별 연산.

리스트와 배열은 구조가 다르다

리스트 [144, 82, 73]



값이 어디에 흩어져 있는지는 그때그때 다르다

배열 `np.array([144, 82, 73])`



같은 자료형이 8바이트씩 붙어 있다

크기와 위치를 미리 아니까, 계산은
C로 짜인 반복문이 한 번에 훑는다

배열이 요구하는 것, 배열이 돌려주는 것

요구: 모든 원소가 같은 자료형이고, 길이가 정해져 있을 것. **보답:** 속도와 짧은 코드.

얼마나 빠른가

노트북 입력 (%timeit은 Jupyter가 주는 측정 도구다)

```
nums = list(range(1_000_000))  
arr = np.arange(1_000_000)  
  
%timeit [x * 2 for x in nums]  
%timeit arr * 2
```

실행 결과

```
17.2 ms ± 217 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)  
511 µs ± 6.56 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
```

ms는 1000분의 1초, µs는 100만분의 1초다. 약 **30배** 차이다. 구체적인 값은 기계마다 다르니, 여러분의 화면에 다른 수가 나오는 것이 정상이다.

확인 질문

여기까지 정리

- 1 노트북을 제출하기 전에 [Restart] 후 [Run All]을 누르는 이유는?
- 2 $[1, 2, 3] + [10, 20, 30]$ 의 결과는? `np.array`로 바꾸면?
- 3 배열이 리스트보다 빠른 이유를 “자료형”과 “메모리”라는 말로 설명해 보자.
- 4 `import numpy as np`에서 `as np`는 무엇을 하는가?

1 Python으로 옮겨오기

2 배열

배열 만들기
파일에서 배열로
인덱싱과 슬라이싱

3 벡터화 연산

4 언어데이터에 적용하기

5 마무리

배열의 네 가지 속성

노트북 입력

```
a = np.array([144, 82, 73, 82, 23])
print(a.shape, a.dtype, a.size, a.ndim)
```

실행 결과

```
(5,) int64 5 1
```

shape 모양. 각 축의 길이를 담은 튜플. (5,)는 “길이 5인 1차원”

dtype 원소의 자료형. int64는 8바이트 정수

size 원소의 총 개수

ndim 축의 개수(차원). 오늘은 대부분 1이다

shape의 (5,)에서 쉼표는 오타가 아니다. 원소가 하나인 튜플의 표기다.

한 배열에는 한 가지 자료형만

노트북 입력

```
np.array([1, 2, 3])  
np.array([1, 2.5])  
np.array([1, 'PEPPER', 3])
```

실행 결과

```
array([1, 2, 3])  
array([1. , 2.5])  
array(['1', 'PEPPER', '3'], dtype='<U21')
```

섞이면 “더 넓은” 자료형으로 끌려간다

정수와 실수가 섞이면 전부 실수가 되고, 문자열이 하나라도 있으면 **전부 문자열**이 된다.
<U21은 “최대 21글자짜리 유니코드 문자열”이라는 뜻이다.

수 1이 문자 '1'로 바뀌면 더 이상 더할 수 없다. dtype은 항상 확인하는 습관을 들인다.

처음부터 만들어 내기

노트북 입력

```
np.arange(5)
np.arange(0, 10, 2)
np.zeros(3)
np.ones(3)
np.linspace(0, 1, 5)
```

실행 결과

```
array([0, 1, 2, 3, 4])
array([0, 2, 4, 6, 8])
array([0., 0., 0.])
array([1., 1., 1.])
array([0. , 0.25, 0.5 , 0.75, 1.  ])
```

읽는 법

- `np.arange`는 Python의 `range`와 같은 규칙이다. 끝값은 포함하지 않는다.
- `np.linspace(시작, 끝, 개수)`는 **끝값을 포함**해 같은 간격으로 나눈다.
- `np.zeros`와 `np.ones`의 결과가 0.처럼 보이는 것은 `float64`이기 때문이다.

np.loadtxt 뜯어보기

노트북 입력

```
words = np.loadtxt('data/apicius/index.tsv', delimiter='\t',  
                   skiprows=1, usecols=2)  
print(words.shape, words.dtype)  
print(words[:5])
```

실행 결과

```
(499,) float64  
[144.  82.  73.  82.  23.]
```

delimiter 필드를 나누는 글자. AWK의 FS에 해당한다

skiprows=1 앞의 한 행을 건너뛴다. AWK의 NR > 1에 해당한다

usecols=2 셋째 열만 읽는다. 0부터 세므로 words 열이다

usecols를 빼면

노트북 입력

```
np.loadtxt('data/apicius/index.tsv', delimiter='\t', skiprows=1)
```

실행 결과

```
ValueError: could not convert string 'FINE SPICED WINE' to float64  
at row 0, column 2.
```

오류 메시지는 읽으라고 있는 것이다

np.loadtxt는 모든 값을 float64로 읽는데, title 열은 수가 아니다.

- AWK는 한 표에 수와 글자가 섞여 있어도 아무렇지 않았다.
- 배열은 그것을 허용하지 않는다. 이것이 속도의 **대가**다.

문자열 열은 따로 읽는다

노트북 입력

```
numbers = np.loadtxt('data/apicius/index.tsv', delimiter='\t',  
                     skiprows=1, usecols=0)  
titles = np.loadtxt('data/apicius/index.tsv', delimiter='\t',  
                    skiprows=1, usecols=1, dtype=str)  
print(titles[:3])
```

실행 결과

```
['FINE SPICED WINE' 'HONEY REFRESHER FOR TRAVELERS' 'ROMAN VERMOUTH']
```

한 표를 배열 **세 개**로 쪼개 들고 있어야 한다. 셋의 순서가 서로 맞다는 것은 우리가 기억할 뿐, 배열은 모른다. 이 불편을 없애는 것이 7강의 데이터프레임이다.

인덱싱: 하나 꺼내기

노트북 입력

```
words[0]  
print(words[0])  
print(words[-1])  
print(words[219])
```

실행 결과

```
np.float64(144.0)  
144.0  
25.0  
0.0
```

두 가지를 눈여겨볼 것

- 번호는 **0부터** 센다. `words[0]` 이 1번 레시피다. 음수는 뒤에서부터다.
- `numpy` 2부터 셀의 마지막 식으로 스칼라를 표시하면 `np.float64(144.0)` 처럼 나온다. `print`로 감싸면 값만 보인다. 앞으로는 `print`를 쓴다.

`words[219]`가 0인 것이 5강의 그 220번 레시피다. 색인과 번호가 하나씩 어긋난다.

슬라이싱: 여러 개 꺼내기

노트북 입력

```
words[:5]  
words[-3:]  
words[::100]  
words[[0, 218, 219]]
```

실행 결과

```
array([144.,  82.,  73.,  82.,  23.])  
array([47., 29., 25.])  
array([144.,  17.,  82.,  69.,   8.])  
array([144.,  11.,   0.])
```

[시작:끝:간격]

문자열과 리스트에서 쓰던 문법 그대로다. 끝값은 포함하지 않는다.

- `words[::100]` 은 100개마다 하나씩 — 색인 0, 100, 200, 300, 400
- `words[[0, 218, 219]]` 처럼 **목록으로** 원하는 자리만 골라도 된다

슬라이스는 복사가 아니다

노트북 입력

```
1 a = np.arange(10)
2 b = a[:3]           # 잘라낸 것처럼 보이지만
3 b[0] = 99          # 원본이 함께 바뀐다
4 print(a)
```

실행 결과

```
[99  1  2  3  4  5  6  7  8  9]
```

뷰(view)와 복사(copy) — 리스트에서 바뀌는 몇 안 되는 규칙

슬라이스는 같은 메모리를 **들여다보는 창**이지 새 배열이 아니다. 원본을 지키려면 `b = a[:3].copy()` 라고 분명히 적는다.

확인 질문

여기까지 정리

- 1 `np.array([1, '2', 3])`의 dtype은 무엇이 되는가? 왜인가?
- 2 `usecols=2`가 셋째 열을 뜻하는 이유는?
- 3 `words[-1]`과 `words[498]`은 같은 값인가?
- 4 `b = a[:3]` 뒤에 `b`를 고치면 `a`는 어떻게 되는가?

1 Python으로 옮겨오기

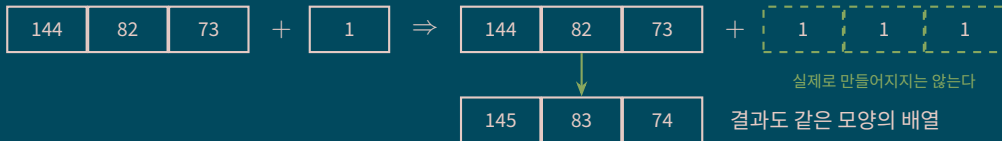
2 배열

3 **벡터화 연산**
원소별 연산
집계
불리언 마스크

4 언어데이터에 적용하기

5 마무리

스칼라 하나가 배열 전체에 퍼진다



브로드캐스팅 (broadcasting)

모양이 다른 두 대상을 계산할 때, 작은 쪽을 큰 쪽의 모양에 **맞추어 늘리는** 규칙. 늘린 배열을 실제로 메모리에 만들지는 않으므로 공짜다.

`words + 1`, `words * 2`, `words / 100` — 모두 499개 전부에 한 번에 적용된다.

언어데이터에서 자주 쓰는 두 가지

정규화: 전체에서 차지하는 비율

```
print(np.round(words[:5] / words.sum() * 100, 2))
```

```
[0.64 0.36 0.32 0.36 0.1 ]
```

각 레시피가 책 전체 22,517단어의 몇 %인지. 길이가 다른 자료를 견줄 때 쓴다. (8강, 13강)

표준화: 평균에서 표준편차 몇 개만큼 떨어져 있나 (z-점수)

```
z = (words - words.mean()) / words.std(ddof=1)
```

```
print(np.round(z[:5], 2))
```

```
[ 2.84  1.06  0.8   1.06 -0.64]
```

1번 레시피는 평균보다 표준편차 2.84개만큼 길다. 단위가 사라져 다른 변수와도 견줄 수 있다.

5강의 값을 되찾기

노트북 입력

```
print(words.size, words.sum())  
print(words.mean(), np.median(words), words.std(ddof=1))  
print(words.min(), words.max())
```

실행 결과

```
499 22517.0  
45.124248496993985 33.0 34.77403524607915  
0.0 273.0
```

5강에서 AWK로 구한 값 — 도구가 바뀌어도 답은 바뀌지 않는다

499 22517 45.1242 중앙값 33 표준편차 34.77 최소·최대 0, 273

집계 함수

하는 일	메서드로	함수로
합계	<code>words.sum()</code>	<code>np.sum(words)</code>
평균	<code>words.mean()</code>	<code>np.mean(words)</code>
표준편차	<code>words.std(ddof=1)</code>	<code>np.std(words, ddof=1)</code>
최소·최대	<code>words.min()</code> , <code>words.max()</code>	<code>np.min</code> , <code>np.max</code>
중앙값	—	<code>np.median(words)</code>
분위수	—	<code>np.percentile(words, 25)</code>

둘의 차이

대부분은 배열에 점을 찍어 쓰는 **메서드**와 `np.`로 부르는 **함수**가 모두 있다. 중앙값과 분위수는 함수만 있다. 외우지 말고, 메서드가 없으면 `np.`를 찾아보면 된다.

ddof: n 이냐 $n - 1$ 이냐

노트북 입력

```
print(words.std(ddof=1))  
print(words.std())
```

실행 결과

```
34.77403524607915  
34.7391740490991
```

numpy의 기본값은 n 이다

ddof는 delta degrees of freedom, 즉 n 에서 빼는 수다. 기본값은 0이다. 5강에서 쓴 $n - 1$ 을 원하면 ddof=1이라고 **매번 적어야** 한다.

어느 쪽을 쓸 것인가

가진 자료 자체를 설명할 때는 n , 더 큰 모집단의 표본으로 볼 때는 $n - 1$ 이다. 언어학 논문의 관행은 $n - 1$ 이므로, 이 수업에서는 ddof=1을 기본으로 한다.

다섯 수치 요약을 한 줄로

노트북 입력

```
print(np.percentile(words, [0, 25, 50, 75, 100]))
```

실행 결과

```
[ 0.  20.  33.  61. 273.]
```

5강의 AWK와 비교

정렬한 뒤 네 자리를 손으로 집어내던 **네 줄짜리** 파이프라인이 한 줄이 되었다.

- `np.percentile`은 목록을 주면 한 번에 여러 분위수를 돌려준다.
- 값이 딱 떨어지지 않으면 이웃한 두 값 사이를 비례로 계산한다. 이 자료에서는 두 방법의 답이 같지만, 언제나 그렇지는 않다.

비교의 결과도 배열이다

노트북 입력

```
print(words[:6])  
print(words[:6] > words.mean())
```

실행 결과

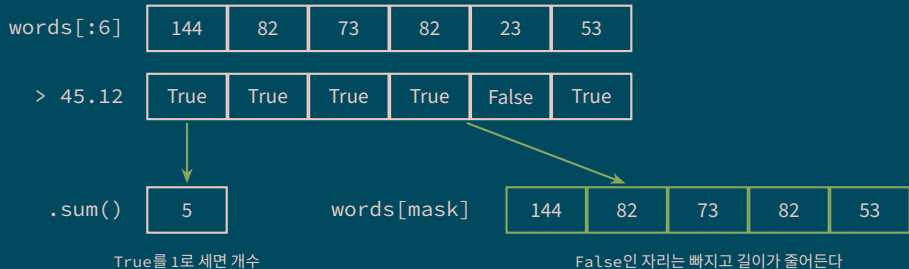
```
[144.  82.  73.  82.  23.  53.]  
[ True  True  True  True False  True]
```

불리언 마스크 (boolean mask)

원본과 **길이가 같은** True/False 배열. “어느 자리가 조건에 맞는가”를 통째로 기록해 둔 것이다.

조건을 한 번 만들어 두면 세는 데도, 골라내는 데도, 다른 배열을 고르는 데도 쓴다.

마스크가 하는 일



한 마스크, 두 가지 쓰임

.sum()은 몇 개인지를, 대괄호 안에 넣으면 어느 것인지를 알려 준다.

세기: True는 1이다

노트북 입력

```
mask = words < words.mean()  
print(mask.sum(), f'{mask.mean():.1%}')
```

실행 결과

```
313 62.7%
```

5강의 그 값 — “레시피의 62.7%가 평균보다 짧다”

- True는 1, False는 0이므로 **합**이 개수, **평균**이 비율이다.
- f'{...:.1%}'는 소수를 백분율로 적는 서식이다. AWK의 printf에 해당한다.

골라내기와 조건 잇기

노트북 입력

```
1 print(words[words > 200])  
2 print((words >= 20) & (words <= 61)).sum()    # 가운데 절반
```

실행 결과

```
[206. 273.]  
258
```

and, or, not은 쓸 수 없다

원소별로 따져야 하므로 &(그리고), |(또는), ~(아니다)를 쓴다. 이들은 비교 연산자보다 먼저 계산되므로 **각 조건을 괄호로 감싼다**. and를 쓰면 `ValueError: The truth value of an array...`가 난다.

마스크로 다른 배열을 고르기

노트북 입력

```
print(numbers[words == 0], titles[words == 0])  
print(numbers[words.argmax()], titles[words.argmax()])
```

실행 결과

```
[220.] ['FOR ROASTS: PEPPER, LOVAGE, CORIANDER, CARRAWAY,']  
186.0 PEAS [supreme style]
```

numpy가 진짜 편해지는 자리 — 길이가 같고 순서가 같으니까

words로 만든 마스크를 numbers와 titles에 그대로 쓴다. argmax는 최댓값 **자리** (색인)를, max는 값을 준다. 5강의 “최솟값과 최댓값은 눈으로 확인한다”가 이제 한 줄이다.

확인 질문

여기까지 정리

- 1 `words > 45`의 결과는 무엇인가? 길이는 몇인가?
- 2 `(words > 45).sum()`과 `(words > 45).mean()`은 각각 무엇을 뜻하는가?
- 3 `words[words > 45]`와 `titles[words > 45]`는 무엇이 같고 무엇이 다른가?
- 4 `words > 20 & words < 61`이라고 쓰면 왜 안 되는가?
- 5 `words.std()`와 `words.std(ddof=1)` 중 5강에서 쓴 것은?

1 Python으로 옮겨오기

2 배열

3 벡터화 연산

4 언어데이터에 적용하기
기술통계량을 직접 만들기
길이와 빈도
2차원 배열

5 마무리

표준편차를 손으로 만들어 보기

노트북 입력 — 5강의 식을 그대로 옮긴다

```
1 d = words - words.mean()           # 편차
2 ss = (d ** 2).sum()                 # 편차 제곱의 합
3 print(np.sqrt(ss / (words.size - 1)))
4 print(words.std(ddof=1))            # 맞는지 확인
```

실행 결과

```
34.77403524607915
34.77403524607915
```

$$s = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n - 1}}$$

수식과 코드가 한 줄씩 대응한다. for문도, 값을 담아 둘 배열도 필요 없다.

편차의 합은 0이어야 하는데

노트북 입력

```
print(d.sum())
```

실행 결과

```
2.2737367544323206e-12
```

0이 아니라 0.0000000000022737이 나왔다 ($e-12$ 는 $\times 10^{-12}$)

컴퓨터는 실수를 정해진 자릿수로만 기억한다. 499번 더하는 동안 아주 작은 오차가 쌓였다.

- 실수끼리는 `==` 대신 `np.isclose(d.sum(), 0)`으로 묻는다.
- 5강의 `words == 0`은 안전했다. 그 값은 단어를 **센** 정수라서다.

3강의 빈도 목록 불러오기

노트북 입력

```
freq = np.loadtxt('data/apicius/word-freq.txt', usecols=0)
vocab = np.loadtxt('data/apicius/word-freq.txt', usecols=1, dtype=str)
print(freq.shape, freq[:5])
print(vocab[:5])
```

실행 결과

```
(2434,) [1265. 1009.  726.  467.  400.]
['AND' 'THE' 'WITH' 'PEPPER' 'BROTH']
```

delimiter를 적지 않으면 공백으로 나눈다. 이 파일은 `sort | uniq -c`가 만든 것이라 앞에 공백이 붙어 있는데, 알아서 처리된다.

numpy가 멈추는 자리

노트북 입력

```
length = np.array([len(w) for w in vocab])  
print(length[:5])
```

실행 결과

```
[3 3 4 6 5]
```

왜 여기만 for가 다시 나오는가

numpy의 벡터화는 **수치** 연산을 위한 것이다.

- 리스트 컴프리헨션으로 길이를 먼저 만들고, 그다음부터 numpy에 넘긴다.
- 여기가 numpy의 경계다. 문자열 열을 편히 다루려면 다른 도구가 필요하다. (7강)

상관계수도 한 줄

노트북 입력

```
print(np.corrcoef(length, freq)[0, 1])  
print(np.corrcoef(length, np.log10(freq))[0, 1])
```

실행 결과

```
-0.13834537169094618  
-0.2949203774523119
```

5강의 awk/corr.awk는 13줄이었다

np.corrcoef는 값 하나가 아니라 **상관행렬**을 돌려준다. 대각선은 자기 자신과의 상관이라 언제나 1이고, 우리가 원하는 값은 [0, 1] 자리다. np.log10도 2,434개 전부에 한 번에 적용된다.

마스크로 갈라 보기

노트북 입력

```
print(f'{length[freq >= 100].mean():.2f}') # 100번 이상 나온 단어  
print(f'{length[freq == 1].mean():.2f}') # 딱 한 번 나온 단어
```

실행 결과

```
4.72  
6.88
```

Zipf의 약어 법칙 — 자주 쓰는 말 4.72글자, 한 번만 쓰인 말 6.88글자

- freq로 만든 마스크를 length에 적용했다. 앞에서 배운 그 기술이다.
- $r = -0.14$ 라는 미묘한 수보다 잘 보인다. 5강의 “나누어 보라”가 이것이다.

한 번만 나온 단어가 절반이다

노트북 입력

```
print((freq == 1).sum(), f' {(freq == 1).mean():.1%}')
```

실행 결과

```
1118 45.9%
```

hapax legomenon — 단어 종류 2,434개 중 1,118개가 딱 한 번 나온다

- 코퍼스를 아무리 키워도 이 비율은 좀처럼 줄지 않는다.
- “이 코퍼스에 없다”가 “그런 말이 없다”는 뜻이 아닌 이유다.
- 8강에서 어휘의 생산성을 재는 재료가 된다.

열을 둘 읽으면

노트북 입력

```
table = np.loadtxt('data/apicius/index.tsv', delimiter='\t',  
                  skiprows=1, usecols=(0, 2))  
print(table.shape, table.ndim)  
print(table[:3])
```

실행 결과

```
(499, 2) 2  
[[ 1. 144.]  
 [ 2.  82.]  
 [ 3.  73.]]
```

shape가 (499, 2), 즉 **499행 2열**이다. 앞이 행, 뒤가 열이다. 대괄호가 두 겹인 것이 2차원이라는 표시다.

axis: 어느 방향으로 접을 것인가

노트북 입력

```
print(table[0, 1])  
print(table[:, 1][:3])  
print(table.mean(axis=0))
```

실행 결과

```
144.0  
[144.  82.  73.]  
[250.          45.1242485]
```

table[0, 1] 0행 1열의 값 하나. 심표로 두 축을 함께 쓴다

table[:, 1] 모든 행의 1열 — 즉 words와 같은 배열

axis=0 행 방향으로 접는다. 열마다 하나씩 값이 남는다

번호의 평균 250은 아무 뜻이 없다. 5강의 척도 이야기 그대로, 계산이 된다고 뜻이 있는 것은 아니다.

그래도 제목은 넣을 수 없다

numpy가 잘하는 것

- 같은 자료형의 수 다발
- 원소별 계산과 집계
- 마스크로 고르고 세기

numpy가 불편한 것

- 얼마다 자료형이 다른 표
- 열을 **이름**으로 부르기
- 문자열 다루기, 결측값

우리가 오늘 겪은 불편

`index.tsv` 한 표를 `numbers`, `titles`, `words` 세 배열로 쪼개 들고, 셋의 순서가 맞기를 **빌면서** 썼다. 한 배열을 정렬하면 그 약속이 깨진다.

표를 표인 채로 다루는 도구가 다음 시간의 `pandas`다.

1 Python으로 옮겨오기

2 배열

3 벡터화 연산

4 언어데이터에 적용하기

5 **마무리**

요약

다음 시간 예고

과제

오늘 쓴 것

하는 일	5강의 AWK	오늘의 numpy
머리글 건너뛰고 읽기	NR > 1, FS	np.loadtxt(skiprows, delimiter)
열 고르기	\$3	usecols=2
평균·표준편차	16줄 스크립트	.mean(), .std(ddof=1)
중앙값·분위수	sort+색인 계산	np.median, np.percentile
조건에 맞는 것 세기	if+변수	(words < 45).sum()
조건에 맞는 것 뽑기	패턴 {동작}	words[words < 45]
상관계수	13줄 스크립트	np.corrcoef(x, y)[0, 1]

바뀐 것과 바뀌지 않은 것

코드는 짧아졌지만 **답은 그대로다**. 도구는 답을 바꾸지 않는다.

오늘 배운 것

노트북

- 셀 단위 실행, 커널, 자동 표시
- 저장소 맨 위에 두어 경로 맞추기
- [Restart] + [Run All]로 재현 확인

배열

- shape, dtype, size, ndim
- 한 배열에 한 자료형
- 슬라이스는 뷰, 복사는 .copy()

벡터화

- 원소별 연산과 브로드캐스팅
- 집계: mean, std, percentile
- 마스크: 세기(sum), 고르기([])

언어데이터

- 5강의 값을 전부 다시 만들었다
- 고빈도어 4.72자, 저빈도어 6.88자
- hapax 1,118개 (45.9%)

다음 시간에 배울 것 (9월 23일 수요일)

제7강 Python 활용 (2) 데이터프레임과 시각화

오늘 배열 셋으로 쪼개 들고 있던 표를 pandas로는 이렇게 읽는다.

```
import pandas as pd

df = pd.read_csv('data/apicius/index.tsv', sep='\t')
print(df.head())
print(df['words'].describe())
```

그때 확인할 것

- 수치 열과 문자 열이 **한 표**에 함께 들어간다.
- describe() 한 줄에 오늘의 기술통계량이 모두 나온다.
- 그리고 5강에서 미뤄 둔 **히스토그램**을 드디어 코드로 그린다.

[숙제03] numpy 연습 (4점)

할 일

노트북 hw03.ipynb를 만들어 아래 넷을 푼다. 각 문항 1점.

- 1 index.tsv에서 z-점수의 절댓값이 3보다 큰 레시피의 **번호와 제목**을 출력한다.
- 2 Q1과 Q3를 **코드로 구해** 그 사이에 있는 레시피가 몇 개인지 센다.
- 3 word-freq.txt에서 길이가 3글자인 단어의 **평균 빈도**를 구한다.
- 4 3번을 3글자부터 10글자까지 for문으로 반복해 5장의 그림 값을 재현한다.

제출

기한 **2026-09-25 12:00 (이번 주 금요일 정오)**. hw03.ipynb를 저장소에 push한다. data/는 커밋하지 않는다. 답을 손으로 적지 말고 **코드가 계산**하게 한다.

[숙제03] 예상 결과와 힌트

1번의 예상 결과 (모두 일곱 개, 가장 긴 186번의 z 는 6.55)

```
141.0 APICIAN DISH  
186.0 PEAS [supreme style]
```

힌트

- 1번: `np.abs(z) > 3`으로 마스크를 만들어 `numbers`와 `titles`에 쓴다.
- 2번: `np.percentile`로 두 값을 받아 `&`로 잇는다. 앞에서 본 258이 나와야 한다.
- 4번: `for k in range(3, 11):` 안에서 `length == k` 마스크를 만든다.

제출 전 [Restart] 후 [Run All]이 끝까지 도는지 확인한다. 질문은 eTL Q&A 게시판에. (답변 마감: 목요일 12:00)